

RINTISAN PARTNER API

SIGNATURE IMPLEMENTATION GUIDE

1 INTRODUCTION

To enhance security, Rintisan API uses **SHA256withRSA** digital signatures with a request body hash.

This ensures that:

- Requests from clients are **authentic** and **untampered**.
- Rintisan callbacks are **validated** by client/partner system to confirm the authenticity.

Request Flow (Partner/Client → Rintisan API)

- Partner/client generates a string to sign by concatenating : **client_id** and **encoded request body**.
- Sign the string using the partner/client private key with **SHA256withRSA**.
- **Base64 encode** the signature.
- Send the generated signature in the **x-signature** header.
- Rintisan API verifies the signature using partner/client public key.

Callback Flow (Rintisan API → Partner/Client)

- Rintisan sign the same way:
 - Signing **client_id** and **encoded request body** with Rintisan private key.
 - **Base64 encode** the signature
- Rintisan will send the signature in the **x-signature** header along with the callback payload
- Partner/client validate the signature using Rintisan public key.

2 SIGNATURE GENERATION

Steps

1. Prepare client_id
2. Minify the JSON request body:
Remove unnecessary spaces, line breaks, and indentation.
3. Generate Body Hash
4. Build the string to sign
5. Sign the data:
Use **SHA256withRSA** with your private key.
6. Base64 encode the signature.
7. Include the signature in **x-signature** header.

PHP Example

```
<?php
$clientId = 'YOUR_CLIENT_ID';
$apiKey = 'YOUR_API_KEY';
$privateKeyPem = file_get_contents(__DIR__ . '/private_key.pem'); // PKCS#1 or PKCS#8 PEM

// Your real request body (as array or raw JSON string). If string, parse + re-encode to minify.
$payload = [
    "partner_trx_id"    => "rin-02",
    "invoice_date"      => "2025-08-07",
    "payment_method"    => "QRIS",
    "amount"            => 20000
];

// Minify JSON
$minifiedBody = json_encode($payload, JSON_UNESCAPED_SLASHES);

// bodyHash = lowercase hex(sha256(minifiedBody))
$bodyHash = strtolower(bin2hex(hash('sha256', $minifiedBody, true)));

// stringToSign = "<client_id>:<bodyHash>"
$stringToSign = $clientId . ':' . $bodyHash;
```

```

// Sign with RSA-SHA256
$pkey = openssl_pkey_get_private($privateKeyPem);
if (!$pkey) { throw new \RuntimeException('Invalid private key'); }

if (!openssl_sign($stringToSign, $rawSig, $pkey, OPENSSL_ALGO_SHA256)) {
    throw new \RuntimeException('Signing failed');
}
$signature = base64_encode($rawSig);

// Example: send with Guzzle (or curl)
$headers = [
    'Content-Type' => 'application/json',
    'x-client_id'   => $clientId,
    'x-api-key'    => $apiKey,
    'x-signature'  => $signature,
];

echo "X-Signature: $signature\n";
echo "Body (minified): $minifiedBody\n";

```

Python Example

```

import json, hashlib, base64
import requests
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding

client_id = "YOUR_CLIENT_ID"
api_key = "YOUR_API_KEY"

# Load RSA private key (PEM)
with open("private_key.pem", "rb") as f:
    private_key = serialization.load_pem_private_key(
        f.read(),
        password=None, # or b"your-password" if encrypted
    )

payload = {

```

```

    "partner_trx_id": "rin-02",
    "invoice_date": "2025-08-07",
    "payment_method": "QRIS",
    "amount": 20000
}

# Minify JSON
minified_body = json.dumps(payload, separators=(',', ':'), ensure_ascii=False)

# bodyHash
body_hash = hashlib.sha256(minified_body.encode("utf-8")).hexdigest().lower()

# stringToSign
string_to_sign = f"{client_id}:{body_hash}"

# Sign (RSA-SHA256, PKCS#1 v1.5)
raw_sig = private_key.sign(
    string_to_sign.encode("utf-8"),
    padding.PKCS1v15(),
    hashes.SHA256()
)
signature = base64.b64encode(raw_sig).decode("utf-8")

headers = {
    "Content-Type": "application/json",
    "x-client_id": client_id,
    "x-api-key": api_key,
    "x-signature": signature,
}

print("X-Signature:", signature)
print("Body (minified):", minified_body)

# Example request:
# r = requests.post("https://your.api/endpoint", headers=headers, data=minified_body,
# timeout=30)
# print(r.status_code, r.text)

```

Node.js Example

```
const fs = require('fs');
const crypto = require('crypto');
const fetch = require('node-fetch'); // or axios

const clientId = 'YOUR_CLIENT_ID';
const apiKey = 'YOUR_API_KEY';
const privateKeyPem = fs.readFileSync('./private_key.pem', 'utf8'); // PKCS#1 or PKCS#8 PEM

const payload = {
  partner_trx_id: "rin-02",
  partner_identifier_id: "RINTISANID",
  invoice_date: "2025-08-07",
  deadline_date: "2025-08-12",
  to_name: "John Doe",
  to_phone: "6281234567890",
  to_email: "john@doe.com",
  payment_method: "QRIS",
  amount: 20000
};

// Minify JSON (parse + stringify ensures normalized formatting)
const minifiedBody = JSON.stringify(JSON.parse(JSON.stringify(payload)));

// bodyHash
const bodyHash = crypto.createHash('sha256').update(minifiedBody,
'utf8').digest('hex').toLowerCase();

// stringToSign
const stringToSign = `${clientId}:${bodyHash}`;

// Sign with RSA-SHA256
const signature = crypto.createSign('RSA-SHA256').update(stringToSign,
'utf8').sign(privateKeyPem).toString('base64');

const headers = {
  'Content-Type': 'application/json',
```

```
'x-client_id': clientId,
'x-api-key': apiKey,
'x-signature': signature
};

console.log('X-Signature:', signature);
console.log('Body (minified):', minifiedBody);

// Example request:
// fetch('https://your.api/endpoint', { method: 'POST', headers, body: minifiedBody })
// .then(r => r.text()).then(console.log).catch(console.error);
```

3 SIGNATURE VALIDATION (CALLBACK)

When receiving a callback from our system:

On every callback:

1. Read header **x-signature**
2. Read the **raw request body** (as text)
3. Minify the JSON (remove whitespace)
4. Compute **bodyHash = sha256(minifiedBody)** → lowercase hex
5. Build **stringToSign = client_id + ":" + bodyHash**
6. Verify RSA-SHA256 over **stringToSign** with **Rintisan public key** (signature is Base64)
7. Reject (HTTP 401/403) if invalid; otherwise return your success JSON.

PHP Validation Example

```
<?php
// config
$CLIENT_ID = 'YOUR_CLIENT_ID';
$RINTISAN_PUBLIC_KEY_PEM = file_get_contents(__DIR__ . '/rintisan_public.pem');
```

```

// 1) Headers & raw body
$signatureB64 = $_SERVER['HTTP_X_SIGNATURE'] ?? ''; // header names become HTTP_...
$rawBody      = file_get_contents('php://input'); // raw request body as sent


// 2) Minify JSON
function minifyJson($body) {
    $arr = json_decode($body, true);
    if ($arr === null && json_last_error() !== JSON_ERROR_NONE) {
        throw new Exception('Invalid JSON body');
    }
    return json_encode($arr, JSON_UNESCAPED_SLASHES);
}

try {
    $minified = minifyJson($rawBody);

    // 3) Body hash (lowercase hex of sha256)
    $bodyHashHex = strtolower(bin2hex(hash('sha256', $minified, true)));

    // 4) Build string to sign
    $stringToSign = $CLIENT_ID . ':' . $bodyHashHex;

    // 5) Verify RSA-SHA256 (signature is Base64)
    $pubKeyId = openssl_pkey_get_public($RINTISAN_PUBLIC_KEY_PEM);
    if (!$pubKeyId) {
        http_response_code(500);
        echo json_encode(["responseCode" => "500", "responseMessage" => "Public key error"]);
        exit;
    }

    $isValid = openssl_verify(
        $stringToSign,
        base64_decode($signatureB64),
        $pubKeyId,
        OPENSSL_ALGO_SHA256
    ) === 1;

    if (!$isValid) {
        http_response_code(401);
    }
}

```

```

        echo json_encode(["responseCode" => "401", "responseMessage" => "Invalid
signature"]);
        exit;
    }

    // Signature OK -> respond success
    header('Content-Type: application/json');
    echo json_encode(["responseCode" => "2005600", "responseMessage" => "Successful"]);
} catch (Exception $e) {
    http_response_code(400);
    echo json_encode(["responseCode" => "400", "responseMessage" => $e->getMessage()]);
}

```

Python Validation Example

```

from flask import Flask, request, jsonify
import base64, json, hashlib
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.exceptions import InvalidSignature

app = Flask(__name__)

CLIENT_ID = "YOUR_CLIENT_ID"
with open("rintisan_public.pem", "rb") as f:
    PUBLIC_KEY = serialization.load_pem_public_key(f.read())

def minify(body_text: str) -> str:
    data = json.loads(body_text)
    return json.dumps(data, separators=(",", ":"), ensure_ascii=False)

@app.post("/callback")
def callback():
    sig_b64 = request.headers.get("x-signature", "")
    raw_body = request.get_data(as_text=True)

    try:
        mini = minify(raw_body)
        body_hash_hex = hashlib.sha256(mini.encode("utf-8")).hexdigest().lower()

```

```

    string_to_sign = f"{CLIENT_ID}:{body_hash_hex}"

    PUBLIC_KEY.verify(
        base64.b64decode(sig_b64),
        string_to_sign.encode("utf-8"),
        padding.PKCS1v15(),
        hashes.SHA256(),
    )
except (InvalidSignature, ValueError, json.JSONDecodeError):
    return jsonify({"responseCode": "401", "responseMessage": "Invalid signature"}), 401

return jsonify({"responseCode": "2005600", "responseMessage": "Successful"}), 200

if __name__ == "__main__":
    app.run(port=5000, debug=True)

```

NodeJs Validation Example

```

const express = require('express');
const crypto = require('crypto');
const fs = require('fs');

const app = express();
// Keep body as raw text to avoid whitespace differences
app.use(express.text({ type: 'application/json' }));

const CLIENT_ID = 'YOUR_CLIENT_ID';
const RINTISAN_PUBLIC_PEM = fs.readFileSync('./rintisan_public.pem', 'utf8');

function minify(bodyText) {
    const obj = JSON.parse(bodyText);
    return JSON.stringify(obj); // default stringifier is compact (no spaces)
}

app.post('/callback', (req, res) => {
    const signatureB64 = req.header('x-signature') || '';
    const rawBody = req.body || '';

    try {

```

```

const mini = minify(rawBody);
const bodyHashHex = crypto.createHash('sha256').update(mini,
'utf8').digest('hex').toLowerCase();
const stringToSign = `${CLIENT_ID}:${bodyHashHex}`;

const verifier = crypto.createVerify('RSA-SHA256');
verifier.update(stringToSign, 'utf8');
const isValid = verifier.verify(RINTISAN_PUBLIC_PEM, signatureB64, 'base64');

if (!isValid) {
  return res.status(401).json({ responseCode: '401', responseMessage: 'Invalid signature' });
}

return res.json({ responseCode: '2005600', responseMessage: 'Successful' });
} catch (e) {
  return res.status(400).json({ responseCode: '400', responseMessage: 'Bad request' });
}
});

app.listen(3000, () => console.log('Callback server running on 3000'));

```

4 PUBLIC KEY EXCHANGE

Client → Rintisan :

Provide your (client) public key (used to verify client requests).

Rintisan → Client:

We (Rintisan) provide our public key (used to verify our callbacks).

5 EXAMPLES

**body request might be different from real API*

Client → Rintisan Payin Create

```
curl --location '{{rintisan_host}}/api/v1/billing/payin/partner/create' \
--header 'Accept: application/json' \
--header 'x-client-id: asd' \
--header 'x-api-key: asd' \
--header 'x-signature:
GvVC19nessSQMCj9K/prwjicbe7u6U8JkOBMqo2DAFVRG1Dg2ShdkCMMFbiziFzRjyguhwqyTLIG1ukKq2j57f
F7xvGaIRwvWnkOluWM80lDSTPwDWRElC1xg879DkFPSkJcg9pfiisEblyzEvBqTl9OyYn0QPN/CCA3lqzL60w=
' \
--header 'Content-Type: application/json' \
--data-raw '{
    "partner_identifier_id": "BILL-PTR-2024-03-11-002",
    "partner_trx_id": "BILL-PTR-2024-06-21-002",
    "from_name": "PT ABC",
    "from_email": "asd@abc.co.id",
    "from_phone": "62813123409878",
    "from_address": null,
    "to_name": "Andi BC",
    "to_email": "abc123@gmail.com",
    "to_phone": "628881858900",
    "to_address": null,
    "invoice_date": "12-07-2024",
    "deadline_date": "29-07-2024",
    "note": null,
    "payment_method": "BLIBLI",
    "amount": 15000,
    "bank_code": "BCA",
    "account_number": "52712345678"
}'
```

Rintisan → Client Callback

```
curl --location '{{partner_host}}/callback_client' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--header 'x-callback-token: asdfbasdfk' \
--header 'x-signature:
nd7B0RtcFDhIFkxEfWJYVkvOvtL0d7m5fYqvct7Gz45toTUIAmalnX4ZXkjsGkDxAL/ybYAjr0sry9eETsZvxOv
n70cVYIFizGZ8jlsPtFUm4povtK46Ye7slIHlkRnb7xdQx9FZlglkCCDUWvs1GKJpUag8ZePjQqDBoNxy16n' \
--data '{
```

```

"timestamp": "2024-08-03 10:30:00",
"paid_at": "2024-08-03 10:35:00",
"partner_trx_id": "TRX123456",
"trx_id": "INT456789",
"payment_code": "PAY001",
"total_amount": 15000,
"status": {
  "code": "200",
  "message": "success"
}
}

```

6 ERROR CODES

Following are error code that might be returned from Rintisan API related to signature

Response Code	Example Response	Description
200	Complete transaction details	Successful Trx & Signature Validation
403	<pre> { "response_code": 403, "message": "Invalid signature" } </pre>	Wrong Signature
400	<pre> { "response_code": 400, "message": "Public key file not found for this partner" } </pre>	Issue in public key : - Invalid content / empty file - Not yet configured in system